

Guía de Operaciones con Cadenas de Texto

Aplicaciones en Análisis Geomático

Viviana Andrea Peña González

22 de mayo de 2026

Tabla de contenidos

1	Introducción	1
1.1	Relevancia en Análisis Espacial	2
2	Ejercicio 1: Limpieza de Topónimos	2
2.1	Contexto	2
2.2	Enunciado	2
2.3	Solución	2
2.4	Respuesta	4
3	Ejercicio 2: Parseo de Coordenadas y Construcción de WKT	4
3.1	Contexto	4
3.2	Enunciado	4
3.3	Solución	4
3.4	Respuesta	6
4	Conclusiones	6
5	Recursos y Referencias	6
5.1	Conclusión	7

1 Introducción

En este documento se presentan soluciones a ejercicios de **operaciones fundamentales con cadenas de texto** orientadas al contexto geomático. En particular, se abordan tareas de:

- **Limpieza de topónimos:** Estandarización de nombres de áreas geográficas
- **Separación de cadenas:** Parseo de coordenadas desde formatos de texto
- **Conversión de tipos:** De texto a valores numéricos

- **Construcción de geometrías:** Generación de formatos WKT (Well-Known Text)

1.1 Relevancia en Análisis Espacial

Estas operaciones son esenciales porque gran parte de los datos geográficos proviene de:

- **Registros manuales:** Inconsistencias en formato y capitalizaciones
- **Sensores y bases de datos heredadas:** Datos con errores e inconsistencias
- **Fuentes heterogéneas:** Información que requiere limpieza y estandarización

El dominio de estas técnicas permite garantizar la calidad de datos antes de análisis más complejos.

2 Ejercicio 1: Limpieza de Topónimos

2.1 Contexto

Se recibe un registro antiguo del IGAC (Instituto Geográfico Agustín Codazzi) con errores de formato del **Parque Nacional Natural Serranía de la Macarena**. El objetivo es estandarizar el nombre para que pueda cruzarse correctamente con sistemas de referencia oficial.

2.2 Enunciado

Dada una cadena de texto con errores de formato, se debe: 1. Eliminar espacios al inicio y final 2. Convertir a formato título 3. Corregir la sigla PNN 4. Calcular la longitud del nombre resultante

2.3 Solución

```
cat("=== Ejercicio 1: Limpieza de Topónimos ===\n\n")
```

```
=== Ejercicio 1: Limpieza de Topónimos ===
```

```
# 1. Registro crudo (con errores)
registro_crudo <- " pnn seRRania de la mACaRena "
cat("Entrada original:\n")
```

Entrada original:

```
cat(sprintf(" '%s'\n", registro_crudo))
```

```

' pnn seRRania de la mACaRena '
cat(sprintf(" Longitud: %d caracteres\n\n", nchar(registro_crudo)))

Longitud: 29 caracteres
# 2. Eliminar espacios al inicio y final
texto_sin_espacios <- trimws(registro_crudo)
cat("Paso 1 - Eliminar espacios laterales:\n")

Paso 1 - Eliminar espacios laterales:
cat(sprintf(" '%s'\n\n", texto_sin_espacios))

'pnn seRRania de la mACaRena'
# 3. Convertir a formato título
library(tools)
texto_titulo <- toTitleCase(tolower(texto_sin_espacios))
cat("Paso 2 - Convertir a formato título:\n")

Paso 2 - Convertir a formato título:
cat(sprintf(" '%s'\n\n", texto_titulo))

'Pnn Serrania De La Macarena'
# 4. Reemplazar Pnn por PNN
registro_oficial <- gsub("Pnn", "PNN", texto_titulo)
cat("Paso 3 - Corregir sigla PNN:\n")

Paso 3 - Corregir sigla PNN:
cat(sprintf(" '%s'\n\n", registro_oficial))

'PNN Serrania De La Macarena'
# 5. Calcular longitud
longitud_registro <- nchar(registro_oficial)

# 6. Reporte final
cat("=== RESULTADO FINAL ===\n")

=== RESULTADO FINAL ===
cat(sprintf("Área protegida estandarizada: %s\n", registro_oficial))

Área protegida estandarizada: PNN Serrania De La Macarena
cat(sprintf("Longitud del nombre: %d caracteres\n", longitud_registro))

Longitud del nombre: 27 caracteres

```

2.4 Respuesta

El área protegida limpia es: **PNN Serrania De La Macarena** y su nombre tiene **27 caracteres**.

3 Ejercicio 2: Parseo de Coordenadas y Construcción de WKT

3.1 Contexto

En análisis geoespacial es común recibir coordenadas almacenadas como texto. Para integrarlas en sistemas de información geográfica (SIG), es necesario convertirlas al estándar **WKT (Well-Known Text)**, que es el formato de serialización de geometrías más ampliamente utilizado.

3.2 Enunciado

Se recibe un mensaje plano enviado por un sensor sísmico ubicado en el Volcán Galeras. El objetivo es extraer el bloque de coordenadas, convertirlo a valores numéricos y generar una geometría WKT POINT con precisión de cuatro decimales. Además, el área afectada debe formatearse con separadores de miles.

3.3 Solución

```
cat("=== Ejercicio 2: Parseo de mensaje de sensor y WKT ===\n\n")
```

```
=== Ejercicio 2: Parseo de mensaje de sensor y WKT ===
```

```
# 1. Mensaje del sensor sísmico
mensaje_sensor <- "ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM"
cat("Mensaje original:\n")
```

Mensaje original:

```
cat(sprintf(" %s\n\n", mensaje_sensor))
```

```
ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM
```

```
# 2. Separar el mensaje por ':' para extraer el bloque de coordenadas
trozos <- strsplit(mensaje_sensor, ":")[[1]]
bloque_coords <- trozos[2]
cat("Bloque de coordenadas extraído:\n")
```

Bloque de coordenadas extraído:

```
cat(sprintf(" %s\n\n", bloque_coords))
```

1.221,-77.359

```
# 3. Separar la longitud y la latitud por ','  
coords <- strsplit(bloque_coords, ",")[[1]]  
lat <- as.numeric(coords[1])  
lon <- as.numeric(coords[2])  
cat("Coordenadas numéricas:\n")
```

Coordenadas numéricas:

```
cat(sprintf(" Latitud: %f\n", lat))
```

Latitud: 1.221000

```
cat(sprintf(" Longitud: %f\n\n", lon))
```

Longitud: -77.359000

```
# 4. Construir el WKT POINT con 4 decimales  
wkt <- sprintf("POINT(%.4f %.4f)", lon, lat)  
cat("WKT generado:\n")
```

WKT generado:

```
cat(sprintf(" %s\n\n", wkt))
```

POINT(-77.3590 1.2210)

```
# 5. Formatear el área afectada con separador de miles  
area_afectada <- 1250000  
area_formateada <- format(area_afectada, big.mark = ".", scientific = FALSE)  
hectareas <- area_afectada / 10000  
cat("Área afectada:\n")
```

Área afectada:

```
cat(sprintf(" %s m²\n", area_formateada))
```

1.250.000 m²

```
cat(sprintf(" %s ha\n\n", format(hectareas, big.mark = ".", scientific = FALSE)))
```

125 ha

```
# 6. Resumen final  
cat("=== RESULTADO FINAL ===\n")
```

=== RESULTADO FINAL ===

```
cat(sprintf("Geometría WKT: %s\n", wkt))
```

Geometría WKT: POINT(-77.3590 1.2210)

```
cat(sprintf("Área afectada: %s m² (%s ha)\n", area_formateada,
           format(hectareas, big.mark = ".", scientific = FALSE)))
```

Área afectada: 1.250.000 m² (125 ha)

3.4 Respuesta

- **WKT generado:** POINT(-77.3590 1.2210)
 - **Área afectada:** 1.250.000 m² (125 ha)
-

4 Conclusiones

Este ejercicio ha demostrado la importancia de las operaciones básicas con cadenas de texto en análisis geomático:

1. **Limpieza de datos:** La estandarización de nombres permite integración confiable entre fuentes heterogéneas de información geográfica.
2. **Parseo de mensajes:** Extraer coordenadas desde un mensaje estructurado requiere separar correctamente bloques y convertir valores a tipo numérico antes de construir geometrías.
3. **Estándares WKT:** El uso del formato WKT garantiza compatibilidad entre diferentes plataformas SIG (ArcGIS, QGIS, PostGIS, etc.).

Estas habilidades son la base para trabajar con datos geomáticos reales, donde la calidad de la información depende de la correcta limpieza y procesamiento en etapas tempranas del análisis.

5 Recursos y Referencias

- **IGAC** (Instituto Geográfico Agustín Codazzi): Base de datos nacional de topónimos
 - **OGC** (Open Geospatial Consortium): Estándar WKT
 - **Función trimws():** Eliminación de espacios en blanco
 - **Función toTitleCase():** Conversión a formato título (paquete tools)
 - **Función strsplit():** Separación de cadenas
 - **Función sprintf():** Formateo de cadenas
 - **Función gsub():** Reemplazo de patrones en texto
-

! Importante

El procesamiento del mensaje permitió extraer correctamente las coordenadas del sensor sísmico y construir la geometría `POINT(-77.3590 1.2210)`. El área afectada fue formateada como `1.250.000 m2`, lo que mejora su legibilidad en reportes y productos espaciales.

i Nota

Sobre el ejercicio 2

En la creación de la geometría WKT, si omitieras el paso de convertir el texto extraído a número decimal y construyeras el WKT directamente concatenando los textos originales, el resultado podría parecer correcto pero no sería confiable para cálculos espaciales posteriores en PostGIS o QGIS.

Sin la conversión numérica, las coordenadas podrían ser interpretadas como texto o contener espacios y formatos inconsistentes, lo que dificulta la validación geométrica, la reproyección y los cálculos de distancia.

i Nota

Si decidiste resolver el Ejercicio 2 usando R, explica por qué fue necesario utilizar `unlist()` o los dobles corchetes `[[1]]` después de ejecutar la función `strsplit()`.

En R, `strsplit()` no devuelve directamente un vector simple, sino una lista. Esa es una diferencia importante frente a Python y Julia, donde la operación equivalente de `split` devuelve una colección más directa para seguir trabajando. Por eso, después de usar `strsplit()` en R es necesario desempaquetar el resultado con `[[1]]` o convertirlo con `unlist()` antes de intentar operar con sus elementos.

Si este paso se omite, el objeto sigue siendo una lista y no un vector de texto simple. En consecuencia, cualquier intento de convertir coordenadas o aplicar cálculos posteriores puede producir errores. Esta es una de las trampas más comunes al trabajar con separación de cadenas en R.

5.1 Conclusión

Este taller permitió comprender que la manipulación de cadenas de texto es una tarea central dentro del flujo de trabajo geomático. La limpieza de topónimos asegura consistencia en cruces y validaciones con bases de datos institucionales, mientras que el parseo de coordenadas y la construcción de geometrías WKT

permiten traducir mensajes crudos o registros textuales en objetos espaciales utilizables.

Además, se evidenció que detalles aparentemente menores, como la eliminación correcta de espacios, el control de decimales o el tipo de salida de una función como `strsplit()`, pueden afectar de manera directa la calidad y la validez de un análisis espacial posterior.